

# AT3B Walkthrough Video — Full Spoken Script with Screen Cues

Target runtime: approximately **12–13 minutes** (under the 15-minute limit).

This document contains only the spoken script and on-screen cues for recording.

---

## Segment 1 — Introduction

### On screen

- Route:  (Home)
- Action: Sit on the home page for a few seconds, no clicking yet.
- Successful result: Home page loads with the game's mode selection visible.

### What to say

*Hi, I'm going to walk you through the final version of my Software Engineering Part B project, Better Bible Guesser. This video covers the features I built and finished during this development phase — the Quest Mode system with Racetrack, Academy and the Marketplace, the updated interface, how data is stored, the security protections I added, and an AI chatbot feature. I'll go through each part of the application and show how it actually works, rather than just describing it.*

### Evidence checkpoint

- Establishes scope: Part B application only, under 15 minutes, sets up all five categories.
- 

## Segment 2 — User Interface: Quest Map and design system

### On screen

- Route: `/quest`
- Action: Click through from Home into Quest Mode; hover/point at the icons for Racetrack, Academy, Marketplace and Castle without clicking Castle yet.
- Successful result: Quest map image loads with four icon buttons and a Scroll balance visible in the header.

## What to say

*This is the Quest Map, the hub for a set of mini-game modes I added this phase. Each icon takes you to a different activity — Racetrack, Academy, and the Marketplace — and along the top you can see my Scroll balance, which is the currency these modes share. I rebuilt the visual design around a consistent set of shared components — a page header, page shell and card styles — so every one of these new screens looks and behaves the same way instead of each being a one-off page. There's also a fourth icon, Castle of Champions, which is still marked as under development, so I won't be demonstrating it today.*

## Evidence checkpoint

- User Interface category: navigation hub, shared design system, honest acknowledgement of an unfinished feature rather than overstating completeness.

---

## Segment 3 — Core Features: Racetrack

### On screen

- Route: `/racetrack`
- Action: Click the Racetrack icon from the Quest Map. Choose the **Easy** set. Play through the 5 verses using a known reference. Reach the results screen.
- Successful result: Results screen shows score/stars and a Scroll reward line.

## What to say

*Racetrack is the main way to earn Scrolls. Every day the app generates three sets of five verses — Easy, Medium and Hard — each with its own time limit and base reward. I'll play the Easy set... [play round] ...and once I finish, the results screen calculates my reward. That reward isn't just the flat base amount — it's weighted by my average score, my average stars, and how much time I had left, so playing well and quickly earns more Scrolls. Only one reward is given per set, per account, per day, so you can't replay the same set to farm currency.*

## Evidence checkpoint

- Core Features category: a complete, reliable workflow with a visible result (Scroll reward), plus an explanation of the underlying reward logic.
- 

## Segment 4 — Core Features: Academy

### On screen

- Route: `/academy`
- Action: Click the Academy icon. Select a lesson set with a working embed. Click "Skip/Continue" past the video (or let it play briefly). Answer the 5 multiple-choice rounds. View local results.
- Successful result: Results stage shows a total score out of the maximum possible.

## What to say

*Academy is a different kind of learning activity. You pick a topic — this one covers [set topic] — watch a short lesson video, and then answer five multiple-choice questions about verses from that passage, instead of typing a reference like in Normal Mode. Answering correctly and quickly earns more points, up to a maximum of 100 per round. One thing I'll be upfront about: Academy currently shows your Scroll balance in the interface, but it doesn't*

*actually award Scrolls for finishing yet — that's a feature I've documented as a known limitation rather than something I've claimed is finished.*

## Evidence checkpoint

- Core Features category: demonstrates a distinct workflow, explains the scoring rule, and honestly flags an incomplete reward hook instead of overclaiming.
- 

## Segment 5 — Core Features: Marketplace and Profile Customisation

### On screen

- Route: `/marketplace` then `/profile`
- Action: Show the 5 daily item slots, purchase one affordable item, confirm the Scroll balance drops. Navigate to Profile → Customize, apply the newly-purchased cosmetic, save.
- Successful result: Marketplace shows the item as "Owned"; profile preview shows the new name effect/nameplate/icon applied.

### What to say

*The Marketplace shows five cosmetic item slots that reset once a day. These aren't randomised every time you open the page — they're calculated deterministically from today's date and my account, so I see the same five items all day even if I navigate away and back. I'll buy this one... [purchase] ...my Scroll balance drops by its price, and it's marked Owned so I can't buy it twice. Now in my Profile, under Customize, that cosmetic is available to actually equip on my display name — so purchasing and customising are two separate steps: buying unlocks it, customising applies it.*

## Evidence checkpoint

- Core Features category: two connected user-facing workflows (purchase → equip) with a visible successful result each.

---

## Segment 6 — User Interface: Settings and accessibility

### On screen

- Route: `/settings`
- Action: Toggle **High Contrast** on, then toggle **Font Scale** up, then Save.
- Successful result: Page colours/contrast and text size visibly change immediately.

### What to say

*The Settings page lets me adjust accessibility and comfort options that are stored on this device. High contrast changes the colour scheme for better readability, and font scale lets me increase text size across the app — you can see the change happen live as I move the slider. These preferences persist after I refresh, because they're saved to local storage on my device rather than needing an account.*

### Evidence checkpoint

- User Interface category: an accessibility feature with a real, visible effect on the application, not just a cosmetic toggle.

---

## Segment 7 — Core Features: Multiplayer lobby mini-game

### On screen

- Route: `/multiplayer` in both windows
- Action: In Window 1, create a lobby as host. In Window 2 (private window), join using the lobby code as a guest.
- Successful result: Window 2 shows "Waiting for host to start the game..." plus a small interactive mini-game.

### What to say

*The existing multiplayer lobby lets friends join with a short code before a game starts. What I added this phase is this small mini-game in the waiting room — while a non-host player waits for the host to start, they can click the moving target for a bit of fun and a locally-saved high score, instead of just staring at a static waiting message. It's intentionally simple and disabled once the real game starts, and the score isn't sent anywhere important — it's just a small quality-of-life addition to an existing screen.*

## Evidence checkpoint

- Core Features category: a substantial, self-contained addition to an existing (pre-AT3B) screen, clearly scoped as new work rather than claiming the whole lobby system as new.

---

## Segment 8 — Data Management

### On screen

- Route: stay on any app page; optionally show the Firebase Console Firestore tab (no private user data visible) for `users`, `sets`, `racetrack_sets`, and `daily_market_seeds` collections at a glance.
- Action: Narrate over the app UI rather than deep-diving into the console.
- Successful result: Collections are visible with anonymised/aggregate-looking documents (no personal data read aloud).

### What to say

*Behind these features, data is split across a few places. Firestore holds structured data: each user's document stores their Scroll balance, their unlocked-items bitmask, their profile customisation, and per-mode statistics like Racetrack challenges completed. Separate collections hold the daily Racetrack sets and the daily Marketplace seed, so the same challenges and shop items are shared by every player on a given day rather than being regenerated per user. The multiplayer lobby system, which already existed*

*before this phase, uses the Realtime Database instead, because it needs fast presence tracking — knowing instantly when a player disconnects. And a few small things, like my settings and the lobby mini-game high score, are only saved in this browser's local storage, not in the cloud.*

## Evidence checkpoint

- Data Management category: distinguishes Firestore vs Realtime Database vs local storage, and explains one real synchronisation reason (presence) for the RTDB choice.
- 

## Segment 9 — Security Features (part 1: authentication and input handling)

### On screen

- Route: `/login` or `/profile`
- Action: Show signing in with a prepared test account (type password quickly, don't narrate it). Then go to the Home page's "Join Game" box and type an invalid game code (e.g. with a symbol) to trigger a validation message.
- Successful result: Signed-in state updates the header; invalid code shows an inline error instead of crashing or submitting.

### What to say

*For accounts, the app uses Firebase Authentication. New visitors are automatically signed in anonymously so they can play straight away, and if they later create a full account with Google or email and password, their anonymous stats are merged into the new account rather than lost. For input, I added a sanitisation service that strips obviously dangerous patterns — like script tags or `javascript:` links — from anything a user types, and validates formats like game codes and verse guesses before they're used. Watch what happens if I type an invalid code here... [type bad code] ...it's rejected with a clear message instead of being sent anywhere.*

## Evidence checkpoint

- Security Features category: demonstrates real authentication behaviour and a real input-validation rejection, addressing account security and basic XSS/format risks.
- 

## Segment 10 — Security Features (part 2: honest gaps and backend protections)

### On screen

- Route: none required — talking segment; optionally show `firestore.rules` and `database.rules.json` briefly as read-only text (not editing).
- Action: No interaction; narrate over a static view of the two rules files or the app.
- Successful result: N/A — informational segment.

### What to say

*I also want to be upfront about where security still needs work, because that's just as important as showing what's finished. My Realtime Database rules require a signed-in user for any read or write, which is appropriate for the multiplayer presence data. My Firestore rules, on the other hand, are currently open to any reader or writer — I used a wide-open rule while actively building and testing this phase, and it has not been locked down yet. That means, right now, anyone with the project's configuration could technically read or write Firestore data directly, bypassing the app entirely. Before any real deployment, this needs proper rules that check authentication and ownership per collection. Similarly, admin access — like the Reset Shop button — is currently only checked in the Angular app itself, against a hard-coded list of admin account IDs; it isn't backed up by matching database rules yet, so it's a client-side convenience rather than a real server-side guarantee. On the other hand, one thing that is properly protected is my AI chatbot's API key — it never reaches the browser at all.*



*The chatbot calls a Cloud Function, and that function holds the Gemini API key on the server side, so it can't be read out of the page's source code.*

## **Evidence checkpoint**

- Security Features category (teacher-required section): explains a real, deployed vulnerability honestly, contrasts it with a correctly-configured protection (RTDB rules, server-side secret), and states what must be fixed before production — without demonstrating an actual exploit against live data.
- 

## **Segment 11 — AI/Advanced Features: Chatbot**

### **On screen**

- Route: any page
- Action: Click the floating chat button, type a short in-game question (e.g. "How do I join multiplayer?"), send it, wait for a reply.
- Successful result: Bot responds with a short, relevant answer within a few seconds.

### **What to say**

*On every page there's a floating chat button. It opens an assistant that can answer short questions about how to play — like how multiplayer works, or what a mode does — and it's specifically instructed not to reveal any Bible verse answers, since that would defeat the point of the game. It also receives some context automatically, like which page I'm on and my basic stats, so its answers can be a little more relevant to what I'm actually doing. If the connection to the server fails, it shows an error message instead of hanging silently — [only mention if this actually happens on camera] ...*

## **Evidence checkpoint**

- AI/Advanced Features category: a working, page-aware AI feature with a visible request/response interaction.
-

## Segment 12 — AI/Advanced Features: Admin controls and deterministic daily content

### On screen

- Route:  (Home, admin panel is visible to admin accounts) — *[Verify before recording: only if signed in with an admin-listed account]*
- Action: Click "Reset Shop", confirm the browser dialog, observe the confirmation alert.
- Successful result: Confirmation alert appears; Marketplace items would reshuffle for all users on next load.

### What to say

*Signed in on an admin account, I also have access to a small admin panel, including this Reset Shop control. Pressing it generates a brand new shared daily seed, which changes which five Marketplace items every player sees for the rest of the day. What's interesting under the hood is that this doesn't rewrite every user's data — because each player's five items are calculated on the fly from their account ID, today's date, and this shared seed using a deterministic random function, changing one small seed value is enough to reshuffle the shop for everyone at once, without a expensive database write per player.*

### Evidence checkpoint

- AI/Advanced Features category: demonstrates an admin-only control and explains the deterministic algorithm that makes daily content generation efficient — evidence of non-trivial logic beyond basic CRUD.

---

## Segment 13 — Testing and final evaluation

### On screen

- Route: none required — talking segment, optionally show a testing results file briefly (e.g.

testing/load/results/\*.summary.txt ) without

reading private data aloud.

- Action: No interaction.
- Successful result: N/A — informational segment.

## What to say

*During testing, I ran a small local load-testing script that opens the app and times things like initial page load and navigating into Quest Mode and the Marketplace, comparing the results against my performance targets. I also manually tested edge cases like invalid game codes, purchasing without enough Scrolls, and replaying a Racetrack set to confirm the one-reward-per-day rule actually holds — all of which behaved correctly and gave a clear error or message instead of failing silently. The biggest limitation I found, and the one I've already talked about, is that my Firestore rules aren't locked down yet — that's the main thing I'd fix next if this were going further. Overall, I think the finished application meets what I set out to build this phase: a working Quest Mode economy, a cleaner interface, and an AI feature layered on top of the existing guessing game, with a few honestly-documented gaps rather than hidden ones.*

## Evidence checkpoint

- Demonstrates acceptance testing evidence and an honest overall evaluation, satisfying the assessment's testing/evaluation expectations without turning into a long report readout.

---

## Segment 14 — Conclusion

### On screen

- Route: /
- Action: Return to the Home page.
- Successful result: N/A — closing shot.

## What to say

*That covers the interface, core features, data storage, security, and the AI feature I built and finished during this development phase. Thanks for watching.*

### **Evidence checkpoint**

- Closes the video within the time limit, reinforcing that only the completed Part B application was shown.